Shaheed Zulfikar Ali Bhutto Institute of Science & Technology

DEPARTEMENT OF ROBOTICS & ARTIFICIAL INTELLIGENCE

Total Marks: 2.5

Obtained Marks: _____

Artificial Neural Network

Lab Task#01

Last date of Submission:13-10-2026

Submitted To: Hassaan Mujtaba

Student Name: Zainab Sohail

Reg. Number: 24108240

```
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, r2_score

student = pd.read_csv("student_performance.csv")

student.columns = student.columns.str.strip().str.lower()

print(student.shape)
print(student.columns.tolist())
print(student.head())
```

```
(1000000, 6)
['student_id', 'weekly_self_study_hours', 'attendance_percentage', 'class_participation', 'total_score', 'grade']
   student_id  weekly_self_study_hours  attendance_percentage  \
0           1                     18.5                   95.6
1           2                     14.0                   80.0
2           3                     19.5                   86.3
3           4                     25.7                   70.2
4           5                     13.4                   81.9

   class_participation  total_score grade
0                  3.8         97.9     A
1                  2.5         83.9     B
2                  5.3        100.0     A
3                  7.0        100.0     A
4                  6.9         92.0     A
```

```
print("Missing values:")
print(student.isnull().sum())

print("\nDataset size:", student.shape)

print("\nTarget column values:")
print(student["total_score"].head())
```

```
Missing values:
student_id                0
weekly_self_study_hours   0
attendance_percentage     0
class_participation       0
total_score               0
grade                     0
dtype: int64

Dataset size: (1000000, 6)
```

```
Target column values:
0     97.9
1     83.9
2    100.0
3    100.0
4     92.0
Name: total_score, dtype: float64
```

```
student["total_score"] = pd.to_numeric(
    student["total_score"], errors="coerce"
)

data = student.dropna(subset=["total_score"]).copy()

X_student = data.drop(
    columns=["total_score", "student_id", "grade"],
    errors="ignore"
)

X_student = pd.get_dummies(X_student, drop_first=True)
X_student = X_student.fillna(0)

y_student = data["total_score"]

print("Features:", X_student.columns.tolist())
print("Total records:", len(data))
```

```
Features: ['weekly_self_study_hours', 'attendance_percentage', 'class_participation']
Total records: 1000000
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X_student,
    y_student,
    test_size=0.2,
    random_state=42
)

print("Training records:", len(X_train))
print("Testing records:", len(X_test))
```

```
Training records: 800000
Testing records: 200000
```

```
student_model = LinearRegression()

student_model.fit(X_train, y_train)
```

```
predictions = student_model.predict(X_test)

print("Student Performance Results")
print("MAE:", round(mean_absolute_error(y_test, predictions), 2))
print("R2 Score:", round(r2_score(y_test, predictions), 2))
print("Model trained successfully!")
```

```
Student Performance Results
MAE: 7.16
R2 Score: 0.66
Model trained successfully!
```

```
%pip install gradio flask
```

```
Requirement already satisfied: gradio in c:\users\zaina\appdata\local\programs\python\python313\lib\site-packages (6.30.0)
Requirement already satisfied: flask in c:\users\zaina\appdata\local\programs\python\python313\lib\site-packages (3.1.3)
Requirement already satisfied: anyio<5.0,>=3.0 in c:\users\zaina\appdata\local\programs\python\python313\lib\site-packages (from gradio) (4.11.0)
Requirement already satisfied: audioop-lts<=0.2.2 in c:\users\zaina\appdata\local\programs\python\python313\lib\site-packages (from gradio) (0.2.2)
Requirement already satisfied: brotli>=1.1.0 in c:\users\zaina\appdata\local\programs\python\python313\lib\site-packages (from gradio) (1.2.0)
Requirement already satisfied: fastapi<1.0,>=0.115.2 in c:\users\zaina\appdata\local\programs\python\python313\lib\site-packages (from gradio) (0.143.0)
Requirement already satisfied: gradio-client==2.7.2 in c:\users\zaina\appdata\local\programs\python\python313\lib\site-packages (from gradio) (2.7.2)
Requirement already satisfied: groovy~=0.1 in c:\users\zaina\appdata\local\programs\python\python313\lib\site-packages (from gradio) (0.1.2)
Requirement already satisfied: hf-gradio<1.0,>=0.4.1 in c:\users\zaina\appdata\local\programs\python\python313\lib\site-packages (from gradio) (0.4.1)
Requirement already satisfied: httpx<1.0,>=0.24.1 in c:\users\zaina\appdata\local\programs\python\python313\lib\site-packages (from gradio) (0.28.1)
Requirement already satisfied: huggingface-hub<3.0,>=1.16.0 in c:\users\zaina\appdata\local\programs\python\python313\lib\site-packages (from gradio) (2.2.
0)
```

```python
import gradio as gr

def predict_student(hours, attendance, participation):
    result = pd.DataFrame([{
        "weekly_self_study_hours": hours,
        "attendance_percentage": attendance,
        "class_participation": participation
    }])

    result = pd.get_dummies(result)
    result = result.reindex(columns=X_student.columns, fill_value=0)

    prediction = student_model.predict(result)[0]

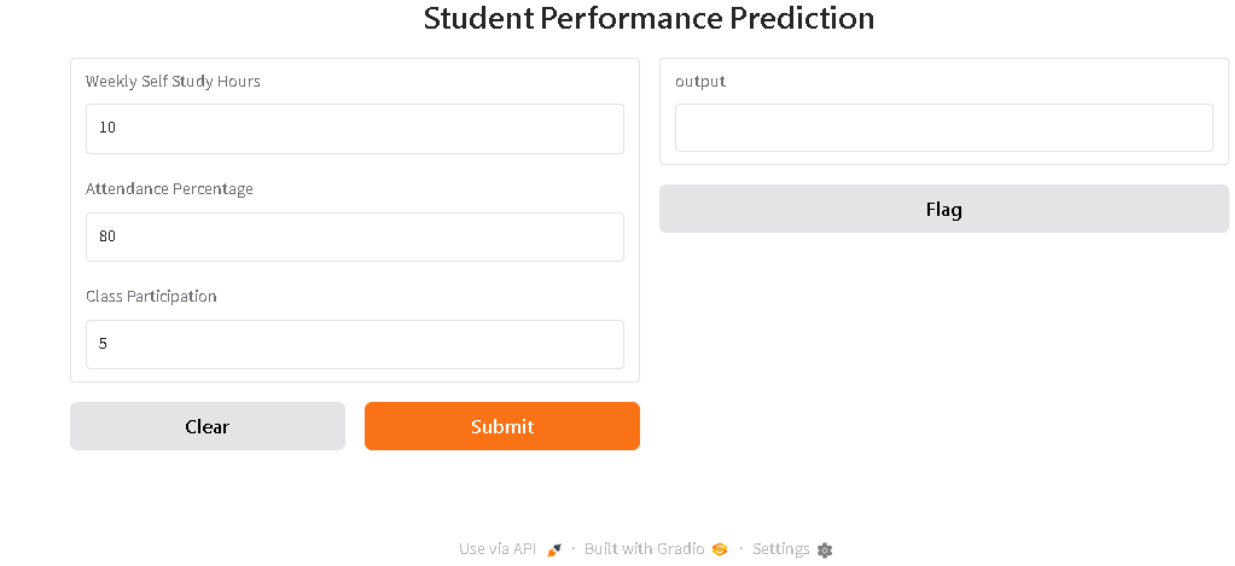
    return f"Predicted Total Score: {prediction:.2f}"

student_app = gr.Interface(
    fn=predict_student,
    inputs=[
        gr.Number(label="Weekly Self Study Hours", value=10),
        gr.Number(label="Attendance Percentage", value=80),
        gr.Number(label="Class Participation", value=5)
    ],
    outputs="text",
    title="Student Performance Prediction"
)

student_app.launch()
```

```
* Running on local URL:  http://127.0.0.1:7861
* To create a public link, set `share=True` in `launch()`.
```

* Running on local URL: http://127.0.0.1:7861
* To create a public link, set `share=True` in `launch()`.



```python
from flask import Flask, request, render_template_string

student_flask = Flask("student_app")

student_html = """
<h2>Student Performance Prediction</h2>
<form method="post">
  Study Hours: <input name="hours" type="number" value="10" step="any"><br><br>
  Attendance: <input name="attendance" type="number" value="80" step="any"><br><br>
  Participation: <input name="participation" type="number" value="5" step="any"><br><br>
  <button type="submit">Predict</button>
</form>
<p>{{ result }}</p>
"""

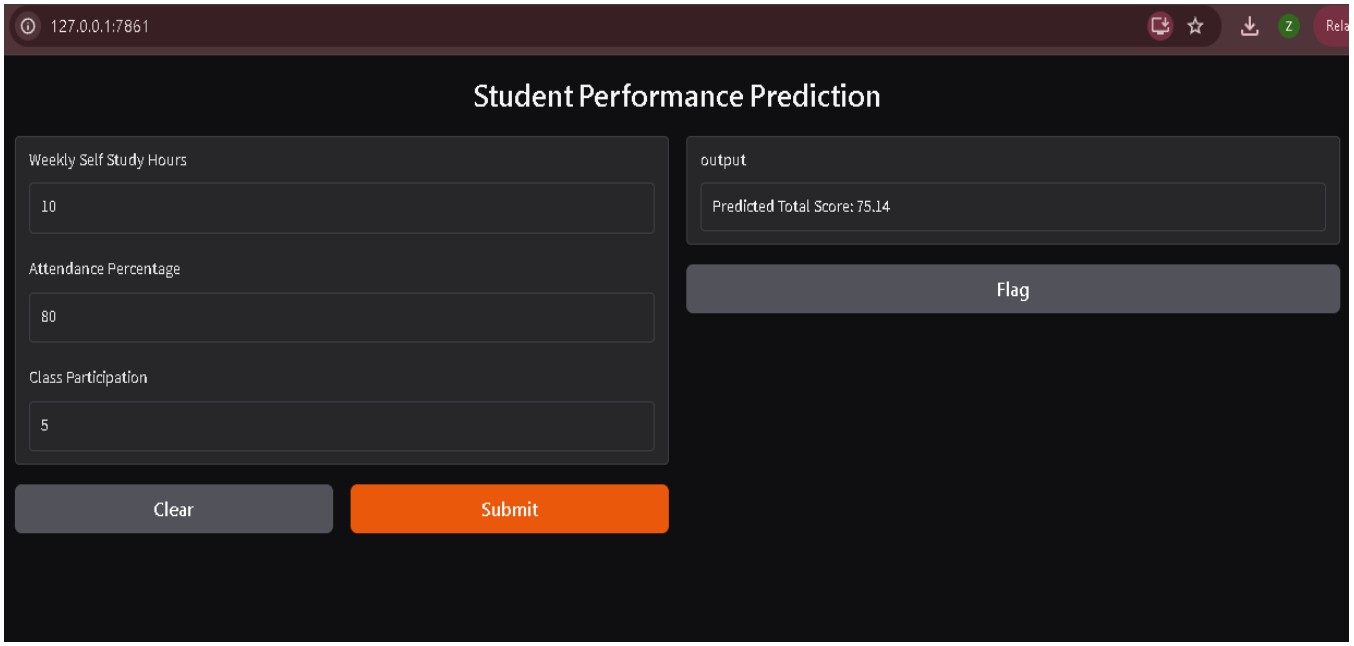
@student_flask.route("/", methods=["GET", "POST"])
def student_home():
    result = ""

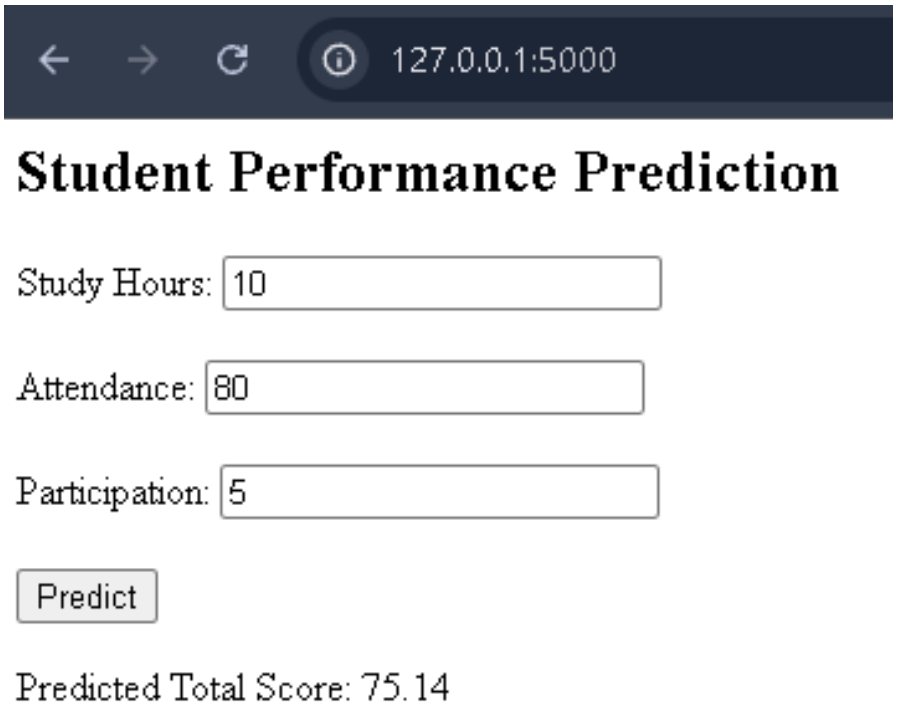
    if request.method == "POST":
        result = predict_student(
            float(request.form["hours"]),
            float(request.form["attendance"]),
            float(request.form["participation"])
        )

    return render_template_string(student_html, result=result)

student_flask.run(port=5000, debug=False, use_reloader=False)
```

```
 * Serving Flask app 'student_app'
 * Debug mode: off
```





DATASET 2:

```
#------------------DATASET 2-------------------------

cars = pd.read_csv("USA_cars_datasets.csv")

cars.columns = cars.columns.str.strip().str.lower()

print(cars.shape)
print(cars.columns.tolist())
print(cars.head())
```

```
(2499, 13)
['unnamed: 0', 'price', 'brand', 'model', 'year', 'title_status', 'mileage', 'color', 'vin', 'lot', 'state', 'country', 'condition']
   unnamed: 0  price      brand    model  year   title_status   mileage  \
0           0   6300     toyota  cruiser  2008  clean vehicle  274117.0
1           1   2899       ford       se  2011  clean vehicle  190552.0
2           2   5350      dodge      mpv  2018  clean vehicle   39590.0
3           3  25000       ford     door  2014  clean vehicle   64146.0
4           4  27700  chevrolet     1500  2018  clean vehicle    6654.0

    color                 vin        lot       state country      condition
0   black    jtezu11f88k007763  159348797  new jersey     usa   10 days left
1  silver    2fmdk3gc4bbb02217  166951262   tennessee     usa    6 days left
2  silver    3c4pdcgg5jt346413  167655728     georgia     usa    2 days left
3    blue    1ftfw1et4efc23745  167753855    virginia     usa  22 hours left
4     red    3gcpcrec2jg473991  167763266     florida     usa  22 hours left
```

```
print(cars["price"].head())
print("Missing prices:", cars["price"].isnull().sum())
```

```
0     6300
1     2899
2     5350
3    25000
4    27700
Name: price, dtype: int64
Missing prices: 0
```

```
cars["price"] = pd.to_numeric(cars["price"], errors="coerce")

car_data = cars.dropna(subset=["price"]).copy()

X_cars = car_data.drop(
    columns=["price", "vin", "lot", "unnamed: 0", "condition"],
    errors="ignore"
)

X_cars = pd.get_dummies(X_cars, drop_first=True)
X_cars = X_cars.fillna(0)

y_cars = car_data["price"]

print("Features:", X_cars.shape)
print("Records:", len(y_cars))
```

```
Features: (2499, 248)
Records: 2499
```

```
X_train_cars, X_test_cars, y_train_cars, y_test_cars = train_test_split(
    X_cars, y_cars, test_size=0.2, random_state=42
)

cars_model = LinearRegression()
cars_model.fit(X_train_cars, y_train_cars)

predictions_cars = cars_model.predict(X_test_cars)

print("USA Cars Results")
print("MAE:", round(mean_absolute_error(y_test_cars, predictions_cars), 2))
print("R2 Score:", round(r2_score(y_test_cars, predictions_cars), 2))
print("Model trained successfully!")
```

```
USA Cars Results
MAE: 4758.56
R2 Score: 0.69
Model trained successfully!
```

```python
import gradio as gr

def predict_car(year, mileage):
    row = pd.DataFrame([{
        "year": year,
        "mileage": mileage
    }])

    row = row.reindex(columns=X_cars.columns, fill_value=0)
    price = cars_model.predict(row)[0]

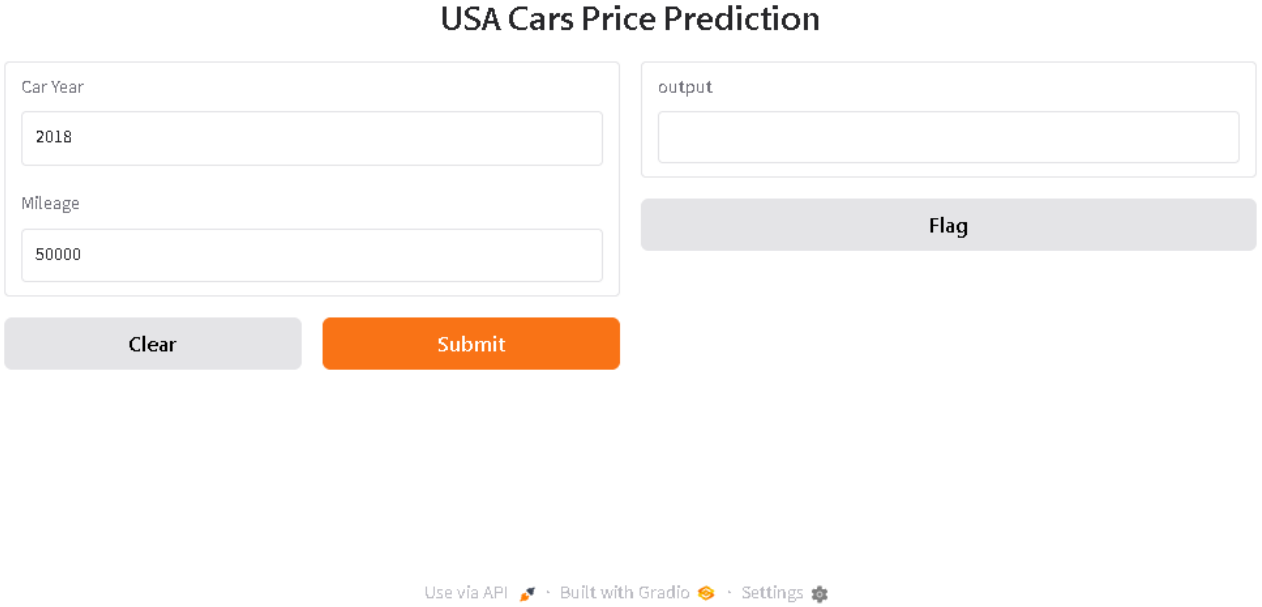
    return f"Predicted Car Price: ${max(0, price):,.2f}"

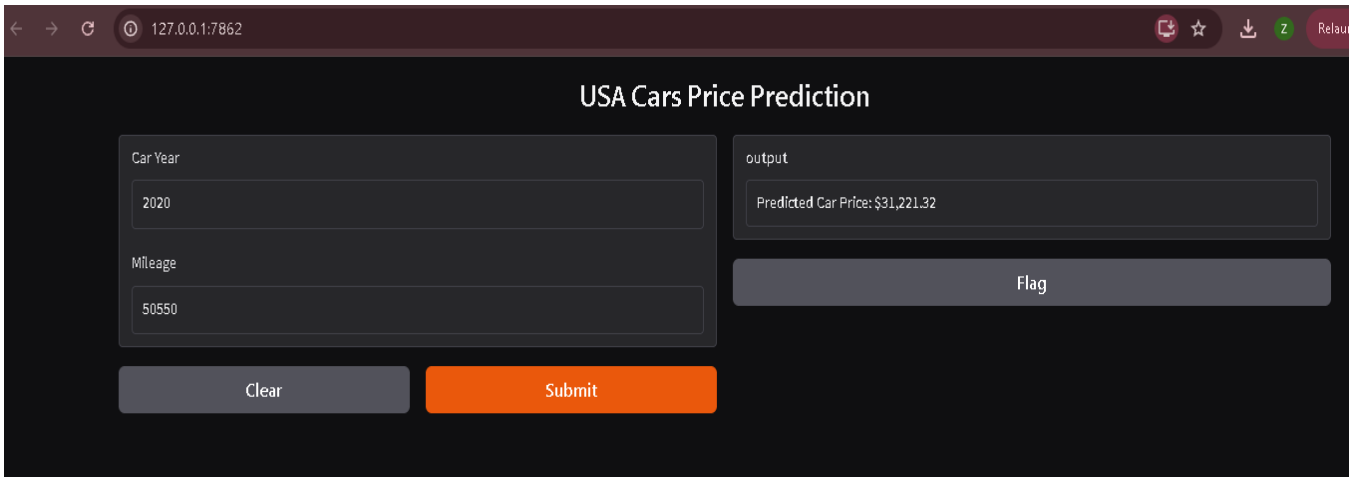
cars_app = gr.Interface(
    fn=predict_car,
    inputs=[
        gr.Number(label="Car Year", value=2018),
        gr.Number(label="Mileage", value=50000)
    ],
    outputs="text",
    title="USA Cars Price Prediction"
)

cars_app.launch()
```

```
* Running on local URL:  http://127.0.0.1:7862
* To create a public link, set `share=True` in `launch()`.
```

* Running on local URL: http://127.0.0.1:7862
* To create a public link, set `share=True` in `launch()`.






```python
from flask import Flask, request, render_template_string

cars_flask = Flask("cars_app")

cars_html = """
<h2>USA Cars Price Prediction</h2>
<form method="post">
  Car Year: <input name="year" type="number" value="2018"><br><br>
  Mileage: <input name="mileage" type="number" value="50000"><br><br>
  <button type="submit">Predict Price</button>
</form>
<p>{{ result }}</p>
"""

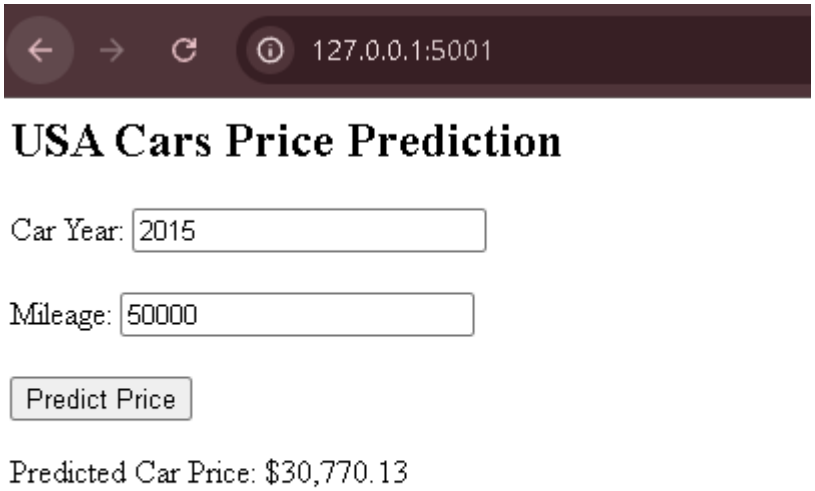
@cars_flask.route("/", methods=["GET", "POST"])
def cars_home():
    result = ""

    if request.method == "POST":
        result = predict_car(
            float(request.form["year"]),
            float(request.form["mileage"])
        )

    return render_template_string(cars_html, result=result)

cars_flask.run(port=5001, debug=False, use_reloader=False)
```

```
 * Serving Flask app 'cars_app'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on http://127.0.0.1:5001
Press CTRL+C to quit
127.0.0.1 - - [12/Oct/2026 00:24:42] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [12/Oct/2026 00:24:42] "GET /favicon.ico HTTP/1.1" 404 -
127.0.0.1 - - [12/Oct/2026 00:24:57] "POST / HTTP/1.1" 200 -
```


127.0.0.1:5001
USA Cars Price Prediction
Car Year: 2015
Mileage: 50000
Predict Price
Predicted Car Price: $30,770.13